ESP32 ENTERTAINMENT BOT

PROJECT BY

-NARAYANAN.N

Software Used

- Arduino 1.8.19 version

Hardware parts used

- Oled
- Esp32[either esp32 wroom (or) esp32-C3]
- Touch sensor
- 3.7V lipo battery

INSTRUCTIONS:

Long press in each mode will show other features.

PROGRAM:

```
//ESP32 CODE
#include <splash.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <Arduino_JSON.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SH110X.h>
#include "time.h"
#include <math.h>
#include <Fonts/FreeSansBold18pt7b.h>
#include <Fonts/FreeSansBold9pt7b.h>
#include <Fonts/FreeSans9pt7b.h>

// ==================================================
// 1. ASSETS & CONFIG
// ==================================================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define SDA_PIN 21 // pin for sda
#define SCL_PIN 22 // pin for scl
#define TOUCH_PIN 4 // pin for touch sensor

Adafruit_SH1106G display(SCREEN_WIDTH, SCREEN_HEIGHT,
&Wire, -1);

// --- WEATHER ICONS ---
const unsigned char bmp_clear[] PROGMEM = { 0x00, 0x00,
0x00, 0x00, 0x00, 0x01, 0x80, 0x00, 0x00, 0x01, 0x80,
0x00, 0x00, 0x01, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00,
0x01, 0x03, 0xc0, 0x80, 0x00, 0x0f, 0xf0, 0x00, 0x00,
0x3f, 0xfc, 0x00, 0x00, 0x7f, 0xfe, 0x00, 0x00, 0xff,
0xff, 0x00, 0x06, 0xff, 0xff, 0x60, 0x06, 0xff, 0xff,
0x60, 0x06, 0xff, 0xff, 0x60, 0x00, 0xff, 0xff, 0x00,
0x3e, 0xff, 0xff, 0x7c, 0x3e, 0xff, 0xff, 0x7c, 0x3e,
0xff, 0xff, 0x7c, 0x00, 0xff, 0xff, 0x00, 0x06, 0xff,
0xff, 0x60, 0x06, 0xff, 0xff, 0x60, 0x06, 0xff, 0xff,
0x60, 0x00, 0xff, 0xff, 0x00, 0x00, 0x7f, 0xfe, 0x00,
```

```
0x00, 0x3f, 0xfc, 0x00, 0x01, 0x0f, 0xf0, 0x80, 0x00,
0x03, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01,
0x80, 0x00, 0x00, 0x01, 0x80, 0x00, 0x00, 0x01, 0x80,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

const unsigned char bmp_clouds[] PROGMEM = { 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x03, 0xe0, 0x00, 0x00, 0x0f, 0xf8,
0x00, 0x00, 0x1f, 0xfc, 0x00, 0x00, 0x3f, 0xfe, 0x00,
0x00, 0x3f, 0xff, 0x00, 0x00, 0x7f, 0xff, 0x80, 0x00,
0xff, 0xff, 0xc0, 0x00, 0xff, 0xff, 0xe0, 0x01, 0xff,
0xff, 0xf0, 0x03, 0xff, 0xff, 0xf8, 0x07, 0xff, 0xff,
0xfc, 0x07, 0xff, 0xff, 0xfc, 0x0f, 0xff, 0xff, 0xfe,
0x0f, 0xff, 0xff, 0xfe, 0x1f, 0xff, 0xff, 0xff, 0x1f,
0xff, 0xff, 0xff, 0x1f, 0xff, 0xff, 0xff, 0x1f, 0xff,
0xff, 0xff, 0x1f, 0xff, 0xff, 0xff, 0x1f, 0xff, 0xff,
0xff, 0x0f, 0xff, 0xff, 0xfe, 0x07, 0xff, 0xff, 0xfc,
0x03, 0xff, 0xff, 0xf8, 0x00, 0xff, 0xff, 0xe0, 0x00,
0x3f, 0xff, 0x80, 0x00, 0x0f, 0xfe, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00 };

const unsigned char bmp_rain[] PROGMEM = { 0x00, 0x00,
0x00, 0x00, 0x00, 0x03, 0xe0, 0x00, 0x00, 0x0f, 0xf8,
0x00, 0x00, 0x1f, 0xfc, 0x00, 0x00, 0x3f, 0xfe, 0x00,
0x00, 0x7f, 0xff, 0x80, 0x00, 0xff, 0xff, 0xc0, 0x01,
0xff, 0xff, 0xf0, 0x03, 0xff, 0xff, 0xf8, 0x07, 0xff,
0xff, 0xfc, 0x0f, 0xff, 0xff, 0xfe, 0x1f, 0xff, 0xff,
0xff, 0x1f, 0xff, 0xff, 0xff, 0x1f, 0xff, 0xff, 0xff,
0x1f, 0xff, 0xff, 0xff, 0x0f, 0xff, 0xff, 0xfe, 0x07,
0xff, 0xff, 0xfc, 0x03, 0xff, 0xff, 0xf8, 0x00, 0xff,
0xff, 0xe0, 0x00, 0x3f, 0xff, 0x80, 0x00, 0x0f, 0xfe,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x60, 0x0c, 0x00,
0x00, 0x60, 0x0c, 0x00, 0x00, 0xe0, 0x1c, 0x00, 0x00,
0xc0, 0x18, 0x00, 0x03, 0x80, 0x70, 0x00, 0x03, 0x80,
0x70, 0x00, 0x03, 0x00, 0x60, 0x00, 0x02, 0x00, 0x40,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

const unsigned char mini_sun[] PROGMEM = { 0x00, 0x00,
0x01, 0x80, 0x00, 0x00, 0x10, 0x08, 0x04, 0x20, 0x03,
0xc0, 0x27, 0xe4, 0x07, 0xe0, 0x07, 0xe0, 0x27, 0xe4,
```

```
0x03, 0xc0, 0x04, 0x20, 0x10, 0x08, 0x00, 0x00, 0x01,
0x80, 0x00, 0x00 };

const unsigned char mini_cloud[] PROGMEM = { 0x00,
0x00, 0x00, 0x00, 0x01, 0xc0, 0x07, 0xe0, 0x0f, 0xf0,
0x1f, 0xf8, 0x1f, 0xf8, 0x3f, 0xfc, 0x3f, 0xfc, 0x7f,
0xfe, 0x3f, 0xfe, 0x1f, 0xfc, 0x0f, 0xf0, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00 };

const unsigned char mini_rain[] PROGMEM = { 0x00, 0x00,
0x00, 0x00, 0x01, 0xc0, 0x07, 0xe0, 0x0f, 0xf0, 0x1f,
0xf8, 0x1f, 0xf8, 0x3f, 0xfc, 0x3f, 0xfc, 0x7f, 0xfe,
0x3f, 0xfe, 0x1f, 0xfc, 0x00, 0x00, 0x44, 0x44, 0x22,
0x22, 0x11, 0x11 };

const unsigned char bmp_tiny_drop[] PROGMEM = { 0x10,
0x38, 0x7c, 0xfe, 0xfe, 0x7c, 0x38, 0x00 };


// --- EMOTION PARTICLES (16x16) ---

const unsigned char bmp_heart[] PROGMEM = { 0x00, 0x00,
0x0c, 0x60, 0x1e, 0xf0, 0x3f, 0xf8, 0x7f, 0xfc, 0x7f,
0xfc, 0x7f, 0xfc, 0x3f, 0xf8, 0x1f, 0xf0, 0x0f, 0xe0,
0x07, 0xc0, 0x03, 0x80, 0x01, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00 };

const unsigned char bmp_zzz[] PROGMEM = { 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x3c, 0x00, 0x0c, 0x00,
0x18, 0x00, 0x30, 0x00, 0x7e, 0x00, 0x00, 0x3c, 0x00,
0x0c, 0x00, 0x18, 0x00, 0x30, 0x00, 0x7c, 0x00, 0x00,
0x00, 0x00, 0x00 };

const unsigned char bmp_anger[] PROGMEM = { 0x00, 0x00,
0x11, 0x10, 0x2a, 0x90, 0x44, 0x40, 0x80, 0x20, 0x80,
0x20, 0x44, 0x40, 0x2a, 0x90, 0x11, 0x10, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00 };


// --- GLOBALS ---

int currentPage = 0;
```

```
bool highBrightness = true;
int tapCounter = 0;
unsigned long lastTapTime = 0;
bool lastPinState = false;
unsigned long pressStartTime = 0;
bool isLongPressHandled = false;
const unsigned long LONG_PRESS_TIME = 800;
const unsigned long DOUBLE_TAP_DELAY = 300;
unsigned long lastPageSwitch = 0;
const unsigned long PAGE_INTERVAL = 8000;

// MOODS
#define MOOD_NORMAL 0
#define MOOD_HAPPY 1
#define MOOD_SURPRISED 2
#define MOOD_SLEEPY 3
#define MOOD_ANGRY 4
#define MOOD_SAD 5
#define MOOD_EXCITED 6
#define MOOD_LOVE 7
#define MOOD_SUSPICIOUS 8
int currentMood = MOOD_NORMAL;

String city = "Delhi"; //city name
const char* countryCode = "IN"; //country code
String apiKey = "00a770fb61f19259e3377ccf39xxxxx";
//openweathermap api
```

```
unsigned long lastWeatherUpdate = 0;
float temperature = 0.0;
float feelsLike = 0.0;
int humidity = 0;
String weatherMain = "Loading";
String weatherDesc = "Wait...";

struct ForecastDay {
  String dayName;
  int temp;
  String iconType;
};
ForecastDay fcast[3];
const char* ssid = "wifi device name change"; // ssid
const char* password = "wifi password";  //password
const char* ntpServer =
"in.pool.ntp.org";//pool.ntp.org
const char* tzString = "IST-5:30"; //timezone

// ==================================================
// 2. ULTRA PRO PHYSICS ENGINE
// ==================================================

struct Eye {
  float x, y;  // Current position (Top-Left)
  float w, h;  // Current size
  float targetX, targetY, targetW, targetH;
```

```
// Pupil Physics (Secondary Motion)
float pupilX, pupilY;
float targetPupilX, targetPupilY;

// Physics constants
float velX, velY, velW, velH;
float pVelX, pVelY;
float k = 0.12;   // Eye Spring
float d = 0.60;   // Eye Damping (Heavier feel)
float pk = 0.08;  // Pupil Spring (Softer/Laggier)
float pd = 0.50;  // Pupil Damping

bool blinking;
unsigned long lastBlink;
unsigned long nextBlinkTime;

void init(float _x, float _y, float _w, float _h) {
  x = targetX = _x;
  y = targetY = _y;
  w = targetW = _w;
  h = targetH = _h;
  pupilX = targetPupilX = 0;
  pupilY = targetPupilY = 0;
  nextBlinkTime = millis() + random(1000, 4000);
}
```

```
  void update() {
    // 1. Main Eye Physics
    float ax = (targetX - x) * k;
    float ay = (targetY - y) * k;
    float aw = (targetW - w) * k;
    float ah = (targetH - h) * k;

    velX = (velX + ax) * d;
    velY = (velY + ay) * d;
    velW = (velW + aw) * d;
    velH = (velH + ah) * d;

    x += velX;
    y += velY;
    w += velW;
    h += velH;

    // 2. Pupil Physics (Dragging behind)
    float pax = (targetPupilX - pupilX) * pk;
    float pay = (targetPupilY - pupilY) * pk;
    pVelX = (pVelX + pax) * pd;
    pVelY = (pVelY + pay) * pd;
    pupilX += pVelX;
    pupilY += pVelY;
  }
};
```

```
Eye leftEye, rightEye;
unsigned long lastSaccade = 0;
unsigned long saccadeInterval = 3000;
float breathVal = 0;

// ====================================================
// 3. LOGIC & NETWORK
// ====================================================
const unsigned char* getBigIcon(String w) {
  if (w == "Clear") return bmp_clear;
  if (w == "Clouds") return bmp_clouds;
  if (w == "Rain" || w == "Drizzle") return bmp_rain;
  return bmp_clouds;
}
const unsigned char* getMiniIcon(String w) {
  if (w == "Clear") return mini_sun;
  if (w == "Rain" || w == "Drizzle" || w ==
"Thunderstorm") return mini_rain;
  return mini_cloud;
}

void updateMoodBasedOnWeather() {
  int m = MOOD_NORMAL;
  if (weatherMain == "Clear") m = MOOD_HAPPY;
  else if (weatherMain == "Rain" || weatherMain ==
"Drizzle") m = MOOD_SAD;
  else if (weatherMain == "Thunderstorm") m =
MOOD_SURPRISED;
```

```
  else if (weatherMain == "Clouds") m = MOOD_NORMAL;
  else if (temperature > 25) m = MOOD_EXCITED;
  else if (temperature < 5) m = MOOD_SLEEPY;
  currentMood = m;
}

void handleTouch() {
  bool currentPinState = digitalRead(TOUCH_PIN);
  unsigned long now = millis();
  if (currentPinState && !lastPinState) {
    pressStartTime = now;
    isLongPressHandled = false;
  } else if (currentPinState && lastPinState) {
    if ((now - pressStartTime > LONG_PRESS_TIME) &&
!isLongPressHandled) {
      lastPageSwitch = now;
      if (currentPage == 0) {
        currentMood++;
        if (currentMood > MOOD_SUSPICIOUS) currentMood
= 0;
        lastSaccade = 0;  // Trigger move
      } else if (currentPage == 1) currentPage = 3;
      else if (currentPage == 2) currentPage = 4;
      isLongPressHandled = true;
    }
  } else if (!currentPinState && lastPinState) {
    if ((now - pressStartTime < LONG_PRESS_TIME) &&
!isLongPressHandled) {
```

```
      tapCounter++;
      lastTapTime = now;
    }
  }
  lastPinState = currentPinState;
  if (tapCounter > 0) {
    if (now - lastTapTime > DOUBLE_TAP_DELAY) {
      lastPageSwitch = now;
      if (tapCounter == 2) {
        highBrightness = !highBrightness;
        display.setContrast(highBrightness ? 255 : 1);
        display.display();
      } else if (tapCounter == 1) {
        if (currentPage == 3) currentPage = 1;
        else if (currentPage == 4) currentPage = 2;
        else {
          currentPage++;
          if (currentPage > 2) currentPage = 0;
        }
      }
      tapCounter = 0;
    }
  }
}

void getWeatherAndForecast() {
  if (WiFi.status() != WL_CONNECTED) return;
```

```
  HTTPClient http;
  String url =
"http://api.openweathermap.org/data/2.5/weather?q=" +
city + "," + countryCode + "&appid=" + apiKey +
"&units=metric";
  http.begin(url);
  if (http.GET() == 200) {
    String payload = http.getString();
    JSONVar myObject = JSON.parse(payload);
    if (JSON.typeof(myObject) != "undefined") {
      temperature = double(myObject["main"]["temp"]);
      feelsLike =
double(myObject["main"]["feels_like"]);
      humidity = int(myObject["main"]["humidity"]);
      weatherMain = (const
char*)myObject["weather"][0]["main"];
      weatherDesc = (const
char*)myObject["weather"][0]["description"];
      weatherDesc[0] = toupper(weatherDesc[0]);
      updateMoodBasedOnWeather();
    }
  }
  http.end();
  url =
"http://api.openweathermap.org/data/2.5/forecast?q=" +
city + "," + countryCode + "&appid=" + apiKey +
"&units=metric";
  http.begin(url);
  if (http.GET() == 200) {
    String payload = http.getString();
```

```
    JSONVar fo = JSON.parse(payload);
    if (JSON.typeof(fo) != "undefined") {
      struct tm t;
      getLocalTime(&t);
      int today = t.tm_wday;
      const char* days[] = { "Sun", "Mon", "Tue",
"Wed", "Thu", "Fri", "Sat" };
      int indices[3] = { 7, 15, 23 };
      for (int i = 0; i < 3; i++) {
        int idx = indices[i];
        fcast[i].temp =
(int)double(fo["list"][idx]["main"]["temp"]);
        fcast[i].iconType = (const
char*)fo["list"][idx]["weather"][0]["main"];
        int nextDayIndex = (today + i + 1) % 7;
        fcast[i].dayName = days[nextDayIndex];
      }
    }
  }
  http.end();
}

// ==================================================
// 4. DRAWING & ANIMATION
// ==================================================

void drawEyelidMask(float x, float y, float w, float h,
int mood, bool isLeft) {
```

```
  int ix = (int)x;
  int iy = (int)y;
  int iw = (int)w;
  int ih = (int)h;
  display.setTextColor(SH110X_BLACK);

  // ANGRY: Sharp slanted cut
  if (mood == MOOD_ANGRY) {
    if (isLeft)
      for (int i = 0; i < 16; i++) display.drawLine(ix,
iy + i, ix + iw, iy - 6 + i, SH110X_BLACK);
    else
      for (int i = 0; i < 16; i++) display.drawLine(ix,
iy - 6 + i, ix + iw, iy + i, SH110X_BLACK);
  }
  // SAD: Inverse slanted cut
  else if (mood == MOOD_SAD) {
    if (isLeft)
      for (int i = 0; i < 16; i++) display.drawLine(ix,
iy - 6 + i, ix + iw, iy + i, SH110X_BLACK);
    else
      for (int i = 0; i < 16; i++) display.drawLine(ix,
iy + i, ix + iw, iy - 6 + i, SH110X_BLACK);
  }
  // HAPPY/LOVE: Cheek push up
  else if (mood == MOOD_HAPPY || mood == MOOD_LOVE ||
mood == MOOD_EXCITED) {
    display.fillRect(ix, iy + ih - 12, iw, 14,
SH110X_BLACK);
```

```
    display.fillCircle(ix + iw / 2, iy + ih + 6, iw /
1.3, SH110X_BLACK);  // Round cut
  }
  // SLEEPY: Heavy lids
  else if (mood == MOOD_SLEEPY) {
    display.fillRect(ix, iy, iw, ih / 2 + 2,
SH110X_BLACK);
  }
  // SUSPICIOUS: One eye squint, one open
  else if (mood == MOOD_SUSPICIOUS) {
    if (isLeft) display.fillRect(ix, iy, iw, ih / 2 -
2, SH110X_BLACK);
    else display.fillRect(ix, iy + ih - 8, iw, 8,
SH110X_BLACK);
  }
}

void drawUltraProEye(Eye& e, bool isLeft) {
  int ix = (int)e.x;
  int iy = (int)e.y;
  int iw = (int)e.w;
  int ih = (int)e.h;

  // 1. Draw Sclera (White base)
  int r = 8;
  if (iw < 20) r = 3;
  display.fillRoundRect(ix, iy, iw, ih, r,
SH110X_WHITE);
```

```
  // 2. Draw Pupil (Black inner eye) - Constrained to
be inside
  // Calculate center of eye
  int cx = ix + iw / 2;
  int cy = iy + ih / 2;

  // Pupil size is proportional to eye size
  int pw = iw / 2.2;
  int ph = ih / 2.2;

  // Apply pupil offset (Gaze)
  int px = cx + (int)e.pupilX - (pw / 2);
  int py = cy + (int)e.pupilY - (ph / 2);

  // Clamp pupil to ensure it doesn't leave the eye
completely
  if (px < ix) px = ix;
  if (px + pw > ix + iw) px = ix + iw - pw;
  if (py < iy) py = iy;
  if (py + ph > iy + ih) py = iy + ih - ph;

  display.fillRoundRect(px, py, pw, ph, r / 2,
SH110X_BLACK);

  // 3. Draw Specular Highlight (The "Glint" of life)
  // A tiny white dot in the top-right of the pupil
  if (iw > 15 && ih > 15) {
```

```
    display.fillCircle(px + pw - 4, py + 4, 2,
SH110X_WHITE);
  }

  // 4. Apply Eyelid Masks (Expressions)
  drawEyelidMask(e.x, e.y, e.w, e.h, currentMood,
isLeft);
}

void updatePhysicsAndMood() {
  unsigned long now = millis();
  breathVal = sin(now / 800.0) * 1.5;  // Breathing
effect

  // --- BLINK LOGIC ---
  if (now > leftEye.nextBlinkTime) {
    leftEye.blinking = true;
    leftEye.lastBlink = now;
    rightEye.blinking = true;
    leftEye.nextBlinkTime = now + random(2000, 6000);
  }
  if (leftEye.blinking) {
    leftEye.targetH = 2;
    rightEye.targetH = 2;  // Close
    if (now - leftEye.lastBlink > 120) {
      leftEye.blinking = false;
      rightEye.blinking = false;
    }
```

```
  }

  // --- SACCADE (Gaze) LOGIC ---
  if (!leftEye.blinking && now - lastSaccade >
saccadeInterval) {
    lastSaccade = now;
    saccadeInterval = random(500, 3000);

    // Pick direction
    int dir = random(0, 10);
    float lx = 0, ly = 0;

    if (dir < 4) {
      lx = 0;
      ly = 0;
    }  // Center
    else if (dir == 4) {
      lx = -6;
      ly = -4;
    }  // TL
    else if (dir == 5) {
      lx = 6;
      ly = -4;
    }  // TR
    else if (dir == 6) {
      lx = -6;
      ly = 4;
```

```
  }  // BL
  else if (dir == 7) {
    lx = 6;
    ly = 4;
  }  // BR
  else if (dir == 8) {
    lx = 8;
    ly = 0;
  }  // R
  else if (dir == 9) {
    lx = -8;
    ly = 0;
  }  // L

  // Move pupil target relative to center
  leftEye.targetPupilX = lx;
  leftEye.targetPupilY = ly;
  rightEye.targetPupilX = lx;
  rightEye.targetPupilY = ly;

  // Move eye container slightly (Head follow)
  leftEye.targetX = 18 + (lx * 0.3);
  leftEye.targetY = 14 + (ly * 0.3);
  rightEye.targetX = 74 + (lx * 0.3);
  rightEye.targetY = 14 + (ly * 0.3);
}
```

```
// --- MOOD SHAPES (Overrides targets) ---
if (!leftEye.blinking) {
  float baseW = 36;
  float baseH = 36;

  // Breathing effect applied to height
  baseH += breathVal;

  switch (currentMood) {
    case MOOD_NORMAL:
      leftEye.targetW = baseW;
      leftEye.targetH = baseH;
      rightEye.targetW = baseW;
      rightEye.targetH = baseH;
      break;
    case MOOD_HAPPY:
    case MOOD_LOVE:
      leftEye.targetW = 40;
      leftEye.targetH = 32;
      rightEye.targetW = 40;
      rightEye.targetH = 32;
      break;
    case MOOD_SURPRISED:
      leftEye.targetW = 30;
      leftEye.targetH = 45;
      rightEye.targetW = 30;
      rightEye.targetH = 45;
```

```
    // Jitter pupil
    leftEye.targetPupilX += random(-1, 2);
    break;
  case MOOD_SLEEPY:
    leftEye.targetW = 38;
    leftEye.targetH = 30;
    rightEye.targetW = 38;
    rightEye.targetH = 30;
    break;
  case MOOD_ANGRY:
    leftEye.targetW = 34;
    leftEye.targetH = 32;
    rightEye.targetW = 34;
    rightEye.targetH = 32;
    break;
  case MOOD_SAD:
    leftEye.targetW = 34;
    leftEye.targetH = 40;
    rightEye.targetW = 34;
    rightEye.targetH = 40;
    break;
  case MOOD_SUSPICIOUS:
    leftEye.targetW = 36;
    leftEye.targetH = 20;  // Left Squint
    rightEye.targetW = 36;
    rightEye.targetH = 42;  // Right Wide
    break;
```

```
    }
  }

  leftEye.update();
  rightEye.update();
}

void drawEmoPage() {
  updatePhysicsAndMood();

  // Draw Floating Particles based on Mood
  if (currentMood == MOOD_LOVE) {
    display.drawBitmap(56, 0, bmp_heart, 16, 16,
SH110X_WHITE);
  } else if (currentMood == MOOD_SLEEPY) {
    display.drawBitmap(110, 0, bmp_zzz, 16, 16,
SH110X_WHITE);
  } else if (currentMood == MOOD_ANGRY) {
    display.drawBitmap(56, 0, bmp_anger, 16, 16,
SH110X_WHITE);
  }

  drawUltraProEye(leftEye, true);
  drawUltraProEye(rightEye, false);
}

// --- STANDARD PAGES ---
void drawForecastPage() {
```

```
  display.fillRect(0, 0, 128, 16, SH110X_WHITE);
  display.setFont(NULL);
  display.setTextColor(SH110X_BLACK);
  display.setCursor(20, 4);
  display.print("3-DAY FORECAST");
  display.setTextColor(SH110X_WHITE);
  display.drawLine(42, 16, 42, 64, SH110X_WHITE);
  display.drawLine(85, 16, 85, 64, SH110X_WHITE);
  for (int i = 0; i < 3; i++) {
    int xStart = i * 43;
    int centerX = xStart + 21;
    display.setFont(NULL);
    String d = fcast[i].dayName;
    if (d == "") d = "Wait";
    display.setCursor(centerX - (d.length() * 3), 20);
    display.print(d);
    display.drawBitmap(centerX - 8, 28,
getMiniIcon(fcast[i].iconType), 16, 16, SH110X_WHITE);
    display.setFont(&FreeSansBold9pt7b);
    int16_t x1, y1;
    uint16_t w, h;

display.getTextBounds(String(fcast[i].temp).c_str(), 0,
0, &x1, &y1, &w, &h);
    display.setCursor(centerX - (w / 2) - 2, 60);
    display.print(fcast[i].temp);
    display.fillCircle(centerX + (w / 2) + 1, 52, 2,
SH110X_WHITE);
```

```
  }
}
void drawClock() {
  struct tm t;
  if (!getLocalTime(&t)) {
    display.setFont(NULL);
    display.setCursor(30, 30);
    display.print("Syncing...");
    return;
  }
  String ampm = (t.tm_hour >= 12) ? "PM" : "AM";
  int h12 = t.tm_hour % 12;
  if (h12 == 0) h12 = 12;
  display.setTextColor(SH110X_WHITE);
  display.setFont(NULL);
  display.setTextSize(1);
  display.setCursor(114, 0);
  display.print(ampm);
  display.setFont(&FreeSansBold18pt7b);
  char timeStr[6];
  sprintf(timeStr, "%02d:%02d", h12, t.tm_min);
  int16_t x1, y1;
  uint16_t w, h;
  display.getTextBounds(timeStr, 0, 0, &x1, &y1, &w,
&h);
  display.setCursor((SCREEN_WIDTH - w) / 2, 42);
  display.print(timeStr);
```

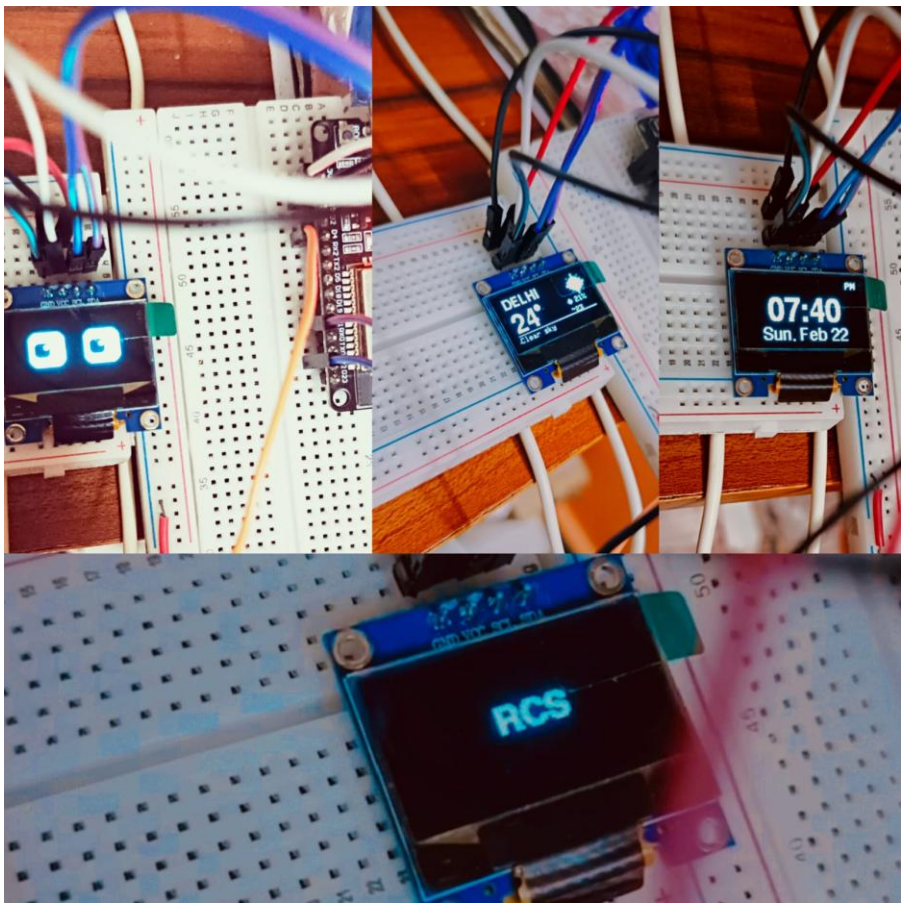
```
  display.setFont(&FreeSans9pt7b);
  char dateStr[20];
  strftime(dateStr, 20, "%a, %b %d", &t);
  display.getTextBounds(dateStr, 0, 0, &x1, &y1, &w,
&h);
  display.setCursor((SCREEN_WIDTH - w) / 2, 62);
  display.print(dateStr);
}
void drawWeatherCard() {
  if (WiFi.status() != WL_CONNECTED) {
    display.setFont(NULL);
    display.setCursor(0, 0);
    display.print("No WiFi");
    return;
  }
  display.drawBitmap(96, 0, getBigIcon(weatherMain),
32, 32, SH110X_WHITE);
  display.setFont(&FreeSansBold9pt7b);
  String c = city;
  c.toUpperCase();
  display.setCursor(0, 14);
  if (c.length() > 9) c = c.substring(0, 8) + ".";
  display.print(c);
  display.setFont(&FreeSansBold18pt7b);
  int tempInt = (int)temperature;
  display.setCursor(0, 48);
  display.print(tempInt);
  int16_t x1, y1;
```

```
  uint16_t w, h;
  display.getTextBounds(String(tempInt).c_str(), 0, 48,
&x1, &y1, &w, &h);
  display.fillCircle(x1 + w + 5, 26, 4, SH110X_WHITE);
  display.setFont(NULL);
  display.drawBitmap(88, 32, bmp_tiny_drop, 8, 8,
SH110X_WHITE);
  display.setCursor(100, 32);
  display.print(humidity);
  display.print("%");
  display.setCursor(88, 45);
  display.print("~");
  display.print((int)feelsLike);
  display.drawLine(0, 52, 128, 52, SH110X_WHITE);
  display.setCursor(0, 55);
  display.print(weatherDesc);
}
void drawWorldClock() {
  time_t now;
  time(&now);
  time_t indiaEpoch = now + (5 * 3600) + (30 * 60);
  time_t sydneyEpoch = now + (11 * 3600);
  struct tm* indiatm = gmtime(&indiaEpoch);
  int i_h = indiatm->tm_hour;
  int i_m = indiatm->tm_min;
  struct tm* sydneytm = gmtime(&sydneyEpoch);
  int s_h = sydneytm->tm_hour;
  int s_m = sydneytm->tm_min;
```

```
  display.fillRect(0, 0, 128, 16, SH110X_WHITE);
  display.setFont(NULL);
  display.setTextColor(SH110X_BLACK);
  display.setCursor(32, 4);
  display.print("WORLD CLOCK");
  display.setTextColor(SH110X_WHITE);
  display.drawLine(64, 18, 64, 54, SH110X_WHITE);
  display.setFont(NULL);
  display.setCursor(16, 22);
  display.print("INDIA");
  display.setFont(&FreeSansBold9pt7b);
  char iStr[10];
  sprintf(iStr, "%02d:%02d", i_h, i_m);
  display.setCursor(5, 46);
  display.print(iStr);
  display.setFont(NULL);
  display.setCursor(78, 22);
  display.print("SYDNEY");
  display.setFont(&FreeSansBold9pt7b);
  char sStr[10];
  sprintf(sStr, "%02d:%02d", s_h, s_m);
  display.setCursor(69, 46);
  display.print(sStr);
  display.setFont(NULL);
  display.setCursor(35, 56);
  display.print("Tap to Exit");
}
```

```
// ==================================================
// 5. BOOT & MAIN
// ==================================================

void playBootAnimation() {
  display.setTextColor(SH110X_WHITE);
  int cx = 64;
  int cy = 32;
  // Expanding Circle Wipe
  for (int r = 0; r < 80; r += 4) {
    display.clearDisplay();
    display.fillCircle(cx, cy, r, SH110X_WHITE);
    display.display();
    delay(10);
  }
  // Inverse Wipe
  for (int r = 0; r < 80; r += 4) {
    display.clearDisplay();
    display.fillCircle(cx, cy, 80, SH110X_WHITE);
    display.fillCircle(cx, cy, r, SH110X_BLACK);
    display.display();
    delay(10);
  }

  // Text Display - "ur name or company name "
  display.setFont(&FreeSansBold9pt7b);
```

```
  String bootText = "NARAYANAN";

  // Calculate width to center it perfectly
  int16_t x1, y1;
  uint16_t w, h;
  display.getTextBounds(bootText, 0, 0, &x1, &y1, &w,
&h);

  display.clearDisplay();
  display.setCursor((SCREEN_WIDTH - w) / 2, 36);
  display.print(bootText);
  display.display();
  delay(2000);
}

void setup() {
  Wire.begin(SDA_PIN, SCL_PIN);
  pinMode(TOUCH_PIN, INPUT);
  display.begin(0x3C, true);
  display.setTextColor(SH110X_WHITE);

  // Init Eyes Center
  leftEye.init(18, 14, 36, 36);
  rightEye.init(74, 14, 36, 36);

  playBootAnimation();
```

```
  display.clearDisplay();
  display.setFont(NULL);
  display.setCursor(40, 30);
  display.print("connecting");
  display.display();
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(200); }
  configTime(0, 0, ntpServer);
  setenv("TZ", tzString, 1);
  tzset();
  getWeatherAndForecast();
  lastWeatherUpdate = millis();
  lastPageSwitch = millis();
}

void loop() {
  unsigned long now = millis();
  handleTouch();
  if (now - lastWeatherUpdate > 600000) {
    getWeatherAndForecast();
    lastWeatherUpdate = now;
  }

  if (currentPage < 3 && now - lastPageSwitch >
PAGE_INTERVAL) {
    currentPage++;
    if (currentPage > 2) currentPage = 0;
```

```
    lastPageSwitch = now;
    lastSaccade = 0;
  }

  display.clearDisplay();
  switch (currentPage) {
    case 0: drawEmoPage(); break;
    case 1: drawClock(); break;
    case 2: drawWeatherCard(); break;
    case 3: drawWorldClock(); break;
    case 4: drawForecastPage(); break;
  }
  display.display();
}
```